

Desarrollo Fullstack – Semana 7

Entity y Repository con JPA

1. Caso seleccionado

Se continuará con una **API REST para la gestión de tareas**.

La aplicación permitirá crear, consultar, actualizar y eliminar tareas. Cada tarea tendrá un identificador, título, descripción y estado.

2. Entity

La entidad Task representa una tarea dentro del sistema y será mapeada a una tabla de la base de datos utilizando JPA.

```
package com.example.tasks.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "tasks")
public class Task {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;

    private String description;

    @Column(nullable = false)
    private String status;

    public Task() {
    }
}
```

```
public Task(String title, String description, String status) {
    this.title = title;
    this.description = description;
    this.status = status;
}

public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getTitle() {
    return title;
}

public void setTitle(String title) {
    this.title = title;
}

public String getDescription() {
    return description;
}

public void setDescription(String description) {
    this.description = description;
}

public String getStatus() {
    return status;
}

public void setStatus(String status) {
    this.status = status;
}
}
```

Explicación del mapeo

- `@Entity`: indica que la clase será una entidad administrada por JPA.
- `@Table(name = "tasks")`: indica que la entidad se almacenará en una tabla llamada `tasks`.
- `@Id`: define el atributo `id` como llave primaria.
- `@GeneratedValue`: permite generar automáticamente el identificador.
- `@Column(nullable = false)`: indica que el atributo no puede tener un valor nulo.

La tabla generada tendría una estructura similar a:

id	title	description	status
1	Realizar taller	Taller de JPA	PENDING
2	Estudiar Spring Boot	Repasar Repository	COMPLETED

3. Repository

Para realizar las operaciones sobre la base de datos se utiliza una interfaz que extiende `JpaRepository`.

```
package com.example.tasks.repository;

import com.example.tasks.entity.Task;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface TaskRepository extends JpaRepository<Task, Long> {

    List<Task> findByStatus(String status);
}
```

Al extender:

```
JpaRepository<Task, Long>
```

se indica que:

- `Task` es la entidad que manejará el `Repository`.

- Long es el tipo de dato de la llave primaria.

Además, se creó la consulta:

```
List<Task> findByStatus(String status);
```

Spring Data JPA interpreta automáticamente el nombre del método y genera una consulta equivalente a buscar todas las tareas que tengan determinado estado.

Por ejemplo:

```
taskRepository.findByStatus("PENDING");
```

permitirá obtener todas las tareas pendientes.

4. Operaciones CRUD

CRUD significa:

Create, Read, Update y Delete.

JpaRepository proporciona estas operaciones sin necesidad de escribir manualmente las consultas SQL.

Create – Crear

Permite registrar una nueva tarea.

```
Task task = new Task(  
    "Realizar taller",  
    "Actividad de JPA",  
    "PENDING"  
);
```

```
taskRepository.save(task);
```

Se utilizaría cuando el usuario quiera agregar una nueva tarea al sistema.

Read – Consultar

Permite consultar la información almacenada.

Para consultar todas las tareas:

```
taskRepository.findAll();
```

Para buscar una tarea por su ID:

```
taskRepository.findById(1L);
```

Para consultar tareas por estado:

```
taskRepository.findByStatus("PENDING");
```

Estas operaciones se utilizarían para mostrar las tareas registradas al usuario.

Update – Actualizar

Para actualizar primero se busca la tarea existente.

```
Task task = taskRepository.findById(1L).orElseThrow();
```

```
task.setStatus("COMPLETED");
```

```
taskRepository.save(task);
```

Esta operación se utilizaría, por ejemplo, cuando el usuario complete una tarea y quiera cambiar su estado de:

PENDING

a:

COMPLETED

Delete – Eliminar

Permite eliminar una tarea de la base de datos utilizando su identificador.

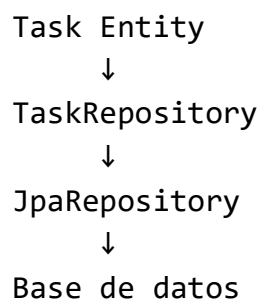
```
taskRepository.deleteById(1L);
```

Se utilizaría cuando el usuario ya no necesite una tarea registrada.

5. Resumen del CRUD

Operación	Método JPA	Uso
Create	<code>save()</code>	Crear una nueva tarea
Read	<code>findAll()</code>	Consultar todas las tareas
Read	<code>findById()</code>	Buscar una tarea por ID
Read	<code>findByStatus()</code>	Buscar tareas por estado
Update	<code>save()</code>	Actualizar una tarea existente
Delete	<code>deleteById()</code>	Eliminar una tarea

6. Flujo



La entidad `Task` define cómo se representan los datos, mientras que `TaskRepository` permite realizar las operaciones CRUD sobre esos datos utilizando Spring Data JPA.

7. Conclusión

El uso de JPA permite relacionar objetos de Java con tablas de una base de datos. La clase `Task` funciona como la entidad que representa las tareas, mientras que `TaskRepository` permite acceder a la información utilizando las operaciones proporcionadas por `JpaRepository`.

Además, Spring Data JPA permite crear consultas personalizadas mediante el nombre de los métodos, como `findByStatus()`, reduciendo la necesidad de escribir consultas SQL manualmente.